

Tutorial: Graph Representations, Traversal, Connected Components, MST & Shortest Path

Learning Outcomes

After studying this tutorial, a student should be able to:	Key techniques covered
Represent a graph using an adjacency matrix and adjacency list	Matrix and list representation
Perform graph traversal using DFS and BFS	Depth-First Search and Breadth-First Search
Explain connected components in an undirected graph	Component discovery using traversal
Understand the ideas behind MST and shortest path algorithms	Prim/Kruskal concepts and Dijkstra concept

1. Graph Representations: Adjacency Matrix and Adjacency List

A graph is a non-linear data structure made of vertices (nodes) and edges (connections). Before running algorithms such as BFS, DFS, shortest path, or minimum spanning tree, we need a way to store the graph in memory.

The two most common representations are the adjacency matrix and the adjacency list.

Adjacency Matrix

- An adjacency matrix is a two-dimensional array of size $V \times V$, where V is the number of vertices.
- If there is an edge from vertex i to vertex j , the value is 1 or the edge weight.
- If there is no edge, the value is 0 or ∞ depending on the context.
- For an undirected graph, the matrix is symmetric.

Example graph: vertices A, B, C, D with edges A-B, A-C, B-D, C-D

	A	B	C	D
A	0	1	1	0
B	1	0	0	1
C	1	0	0	1
D	0	1	1	0

Advantages of adjacency matrix:

- Very simple to implement.

- Checking whether an edge exists between two vertices is $O(1)$.
- Useful for dense graphs.

Disadvantages of adjacency matrix:

- Requires $O(V^2)$ space.
- Not memory-efficient for sparse graphs.

Adjacency List

- An adjacency list stores the list of neighbors for each vertex.
- It can be implemented using arrays of linked lists, vectors, or dynamic arrays.
- In a weighted graph, each entry stores the neighbor and the weight.

For the same example graph:

Vertex	Adjacent vertices
A	B, C
B	A, D
C	A, D
D	B, C

Advantages of adjacency list:

- Needs only $O(V + E)$ space.
- Best for sparse graphs.
- Very convenient for traversing neighbors.

Quick comparison

Feature	Adjacency Matrix	Adjacency List	Best used when
Space	$O(V^2)$	$O(V + E)$	Depends on density
Edge lookup	Very fast	Usually slower	Matrix
Neighbor traversal	Less convenient	Very convenient	List
Graph type	Dense	Sparse	—

2. Graph Traversal: DFS and BFS

Graph traversal means visiting all vertices in a systematic order. The two main traversal methods are DFS and BFS.

Depth-First Search (DFS)

- DFS goes as deep as possible along one path before backtracking.
- It can be implemented using recursion or an explicit stack.

Basic idea of DFS:

1. Start from a source vertex.
2. Mark it visited.
3. Visit one unvisited neighbor.
4. Continue deeper until no new neighbor remains.
5. Backtrack and continue.

Breadth-First Search (BFS)

- BFS visits vertices level by level.
- It is implemented using a queue.

Basic idea of BFS:

6. Start from a source vertex and enqueue it.
7. Mark it visited.
8. Dequeue a vertex.
9. Visit all unvisited neighbors and enqueue them.
10. Repeat until the queue becomes empty.

Worked example

Adjacency list: A: B, C B: D, E C: F D: — E: F F: —

Method	Possible visit order from A	Main data structure
DFS	A, B, D, E, F, C	Stack / recursion
BFS	A, B, C, D, E, F	Queue

Key difference:

- DFS explores depth first; BFS explores level by level.
- BFS is especially useful for shortest path in an unweighted graph.
- Both DFS and BFS take $O(V + E)$ time with adjacency lists.

3. Connected Components

In an undirected graph, a connected component is a maximal set of vertices such that every vertex is reachable from every other vertex in that set.

- If the whole graph is one reachable group, the graph is connected.
- If the graph breaks into separate groups, each group is a connected component.

Example

Vertices {A, B, C, D, E, F} with edges {A–B, B–C, D–E} have:

- Component 1: {A, B, C}
- Component 2: {D, E}
- Component 3: {F}

How to find connected components

11. Mark all vertices unvisited.
12. Choose an unvisited vertex and run DFS or BFS.
13. All reached vertices form one component.
14. Repeat until every vertex is visited.

4. Minimum Spanning Tree (MST)

A minimum spanning tree is defined for a connected, undirected, weighted graph.

- A spanning tree connects all vertices.
- It contains exactly $V - 1$ edges.
- It contains no cycle.
- Among all spanning trees, the MST has the minimum total edge weight.

Common MST algorithms

Algorithm	Main idea	Common data structure
Prim's algorithm	Grow the tree by repeatedly taking the minimum edge that expands the tree	Priority queue
Kruskal's algorithm	Sort edges by weight and choose the next smallest edge that does not form a cycle	Disjoint set / union-find

5. Shortest Path

The shortest path problem asks for the minimum distance from a source vertex to another vertex, or to all vertices.

Common shortest path situations

Situation	Suitable method	Key idea
Unweighted graph	BFS	Fewest edges gives shortest distance
Weighted graph with non-negative weights	Dijkstra's algorithm	Repeatedly choose the nearest unsettled vertex
Graphs with negative weights	Bellman-Ford (advanced topic)	Relax all edges repeatedly

MST versus shortest path

- MST minimizes the total cost to connect all vertices.
- Shortest path minimizes the cost between a source and a destination.
- These two goals are different.

6. Summary

- Adjacency matrix uses a $V \times V$ table; adjacency list stores neighbors for each vertex.
- DFS uses depth-first exploration; BFS uses level-order exploration.
- Connected components are separate reachable groups.
- MST connects all vertices with minimum total cost.
- Shortest path finds the minimum-distance route from source to destination.